

Jira Task 03 — Database-per-Service & Data Ownership.

Step 1 — Learn

1. Shared Database

A **shared database** is a database that is used by multiple services or modules.

Example:

User Service ---- ---|
Product Service ---|→ One Shared Database
Order Service ----- |

2. Database-per-Service

In this approach, **each microservice has its own database.**

Example:

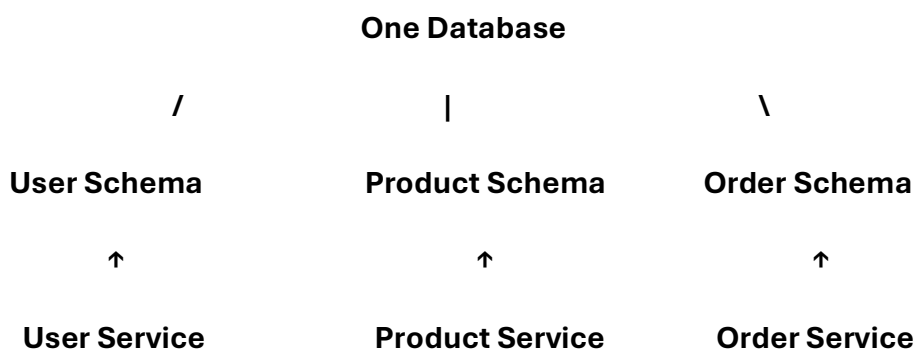
User Service → User Database
Product Service → Product Database
Order Service → Order Database

Each service is responsible for its own data.

3. Schema-per-Service

Here, services can use the **same database server**, but each service has its **own separate schema**.

Example:



4. Data Ownership

Data ownership means that each service is responsible for creating, updating, and managing its own data.

Example:

- **User Service** → owns user data
- **Product Service** → owns product data
- **Order Service** → owns order data

The Order Service should **not directly modify the User Service's database**. If it needs user information, it should communicate with the User Service through an API or another appropriate service-to-service mechanism.

Step 2 - Create Three Services

1. User Service
2. Order Service
3. Payment Service

1 User Service

Responsible for user information.

Example:

User Service

↓

User Database

↓

users table

It can handle:

- Create user

- Get user
- Update user
- Delete user

2 Order Service

Responsible for orders.

Order Service



Order Database



orders table

It can handle:

- Create order
- View order
- Update order
- Cancel order

3 Payment Service

Responsible for payments.

Payment Service



Payment Database



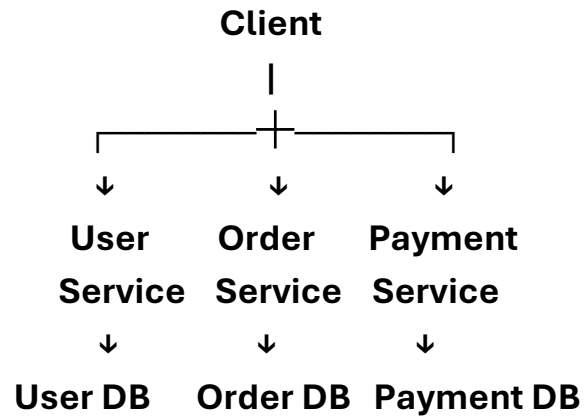
payments table

It can handle:

- Make payment

- Check payment status
- Payment history

Overall architecture



Step 3 — Database/Schema Design

I recommend using **MySQL** for this task.

Create three separate databases:

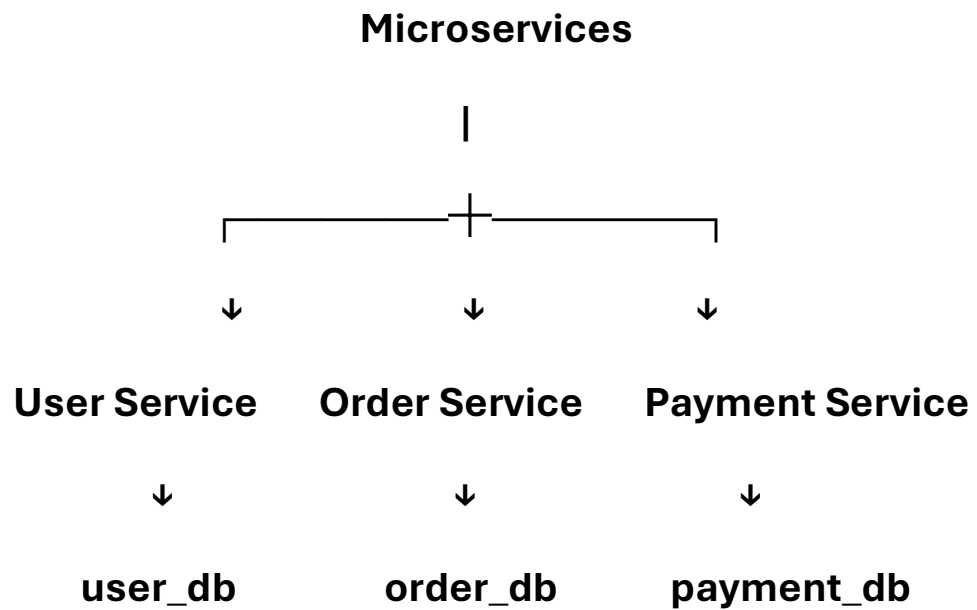
SQL:

```
CREATE DATABASE user_db;
```

```
CREATE DATABASE order_db;
```

```
CREATE DATABASE payment_db;
```

Your architecture will be:



1. User Service

Database:

user_db

The User Service owns all user-related data.

Example:

user_db

└─ users

2. Order Service

Database:

order_db

The Order Service owns order-related data.

order_db
└─ orders

3. Payment Service

Database:

payment_db

The Payment Service owns payment-related data.

payment_db
└─ payments

Step 4 — Create Entities/Tables

For the microservices application, each service should have its **own database/schema and tables**.

1. User Service

Create a users table:

Column	Type	Description
id	BIGINT	Primary key
name	VARCHAR	Username
email	VARCHAR	User email
password	VARCHAR	User password
created_at	TIMESTAMP	Account creation time

2. Order Service

Create two tables:

Orders

Column	Type	Description
id	BIGINT	Primary key
user_id	BIGINT	ID of the user
total_amount	DECIMAL	Order total
status	VARCHAR	Order status
created_at	TIMESTAMP	Order date

order_items

Column	Type	Description
id	BIGINT	Primary key
order_id	BIGINT	Order ID
product_id	BIGINT	Product ID
quantity	INT	Quantity
price	DECIMAL	Product price

3. Payment Service

Create a payments table:

Column	Type	Description
id	BIGINT	Primary key
order_id	BIGINT	Order ID
amount	DECIMAL	Payment amount
payment_method	VARCHAR	Card/UPI/etc.
status	VARCHAR	Payment status
created_at	TIMESTAMP	Payment date

User Service: users

Order Service: orders, order_items

Payment Service: payments

Each microservice owns and manages its respective tables/database.

Services should not directly access another service's database.

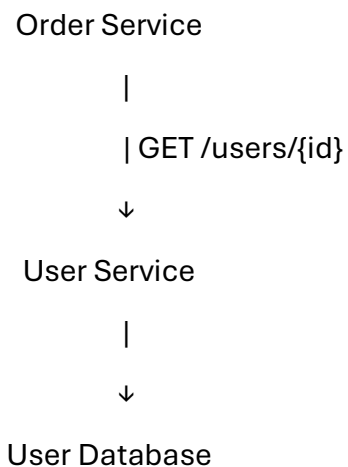
Step 5 — Do Not Directly Access Another Service's Database

For our application:

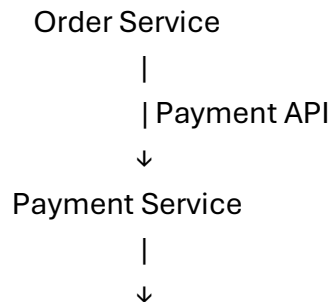
- **User Service** → accesses only the users table/database.
- **Order Service** → accesses only the orders and order_items tables/database.
- **Payment Service** → accesses only the payments table/database.

If the **Order Service** needs user information, it should **not** directly query the User Service database. Instead, it should call the **User Service API**.

Example:



Similarly:



Payment Database

Ensure database isolation between microservices.

Each service must access and modify only its own database. A service must not directly query or modify another service database.

Required information from another service must be obtained through its APIs.

Step 6 —

Design APIs for accessing required information

The purpose of Step 6 is to make sure **services get information from another service through APIs**, instead of directly accessing that service's database.

For your project, you have:

User Service

Order Service

Payment Service

1. User Service APIs

The User Service owns **user_db**.

Example APIs:

POST /api/users

GET /api/users/{id}

PUT /api/users/{id}

DELETE /api/users/{id}

For example, if Order Service needs user information:

Order Service

|

| GET /api/users/101

↓

User Service

|

↓

user_db

The Order Service does not directly connect to **user_db**.

2. Order Service APIs

The Order Service owns **order_db**.

POST /api/orders

GET /api/orders/{id}

GET /api/orders/user/{userId}

PUT /api/orders/{id}

Example:

Client

↓

Order Service

↓

order_db

3. Payment Service APIs

The Payment Service owns **payment_db**.

POST /api/payments

GET /api/payments/{id}

GET /api/payments/order/{orderId}

Example:

Order Service

|

| POST /api/payments

↓

Payment Service

↓

payment_db

Designed REST APIs for communication between microservices.

User Service exposes APIs for user information,

Order Service exposes APIs for order management, and

Payment Service exposes APIs for payment operations.

Services access required information through APIs instead of directly accessing another service database.

This demonstrates data ownership and proper microservices of communication.

Step 7 — Document Data Ownership

1. Who owns each database?

Service	Database	Owner
User Service	user_db	User Service
Order Service	order_db	Order Service
Payment Service	payment_db	Payment Service

So:

User Service → user_db

Order Service → order_db

Payment Service → payment_db

2. Who can modify the data?

Only the owning service should create, update, or delete its data.

User Service → Can modify user data 

Order Service → Can modify order data 

Payment Service → Can modify payment data 

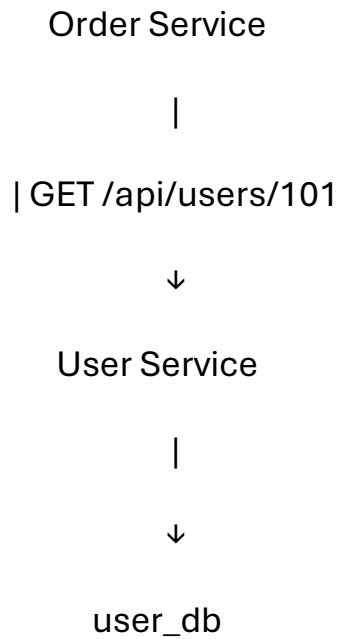
For example:

Order Service should **not** directly modify user_db.

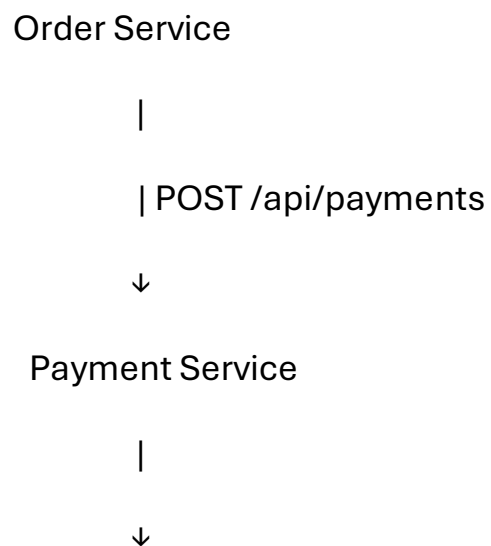
If Order Service needs user information, it should call the User Service API.

3. How do services exchange information?

Services communicate through **REST APIs**.



For payment:



payment_db

Data Ownership Documentation

- User Service owns user_db and is responsible for user data.
- Order Service owns order_db and is responsible for order data.
- Payment Service owns payment_db and is responsible for payment data.
- Only the owning service can create, update, or delete its data.
- Services must not directly access another service database.
- Services exchange information through REST APIs.

Step 8 — Test the Architecture

The purpose of Step 8 is to verify that your **User, Order, and Payment microservices** work correctly and follow the database-ownership rules.

1. Test User Service

Make sure the User Service can create and retrieve users.

POST /api/users

GET /api/users/{id}

Example:

POST /api/users

```
{  
  "name": "Raju",  
  "email": "raju@example.com"  
}
```

Expected result: User is saved in **user_db**

2. Test Order Service

POST /api/orders

GET /api/orders/{id}

Create an order and verify that it is stored in **order_db**.

3. Test Payment Service

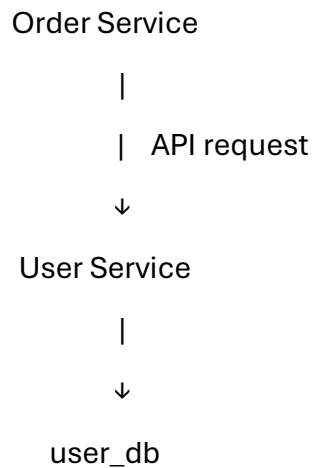
POST /api/payments

GET /api/payments/{id}

Create a payment and verify that it is stored in **payment_db**.

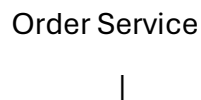
4. Test service-to-service communication

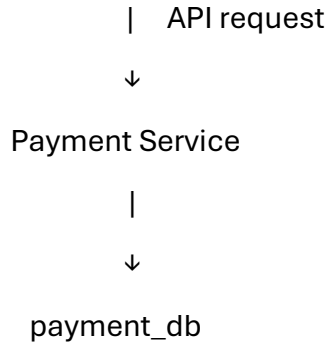
For example:



The important thing is that **Order Service must not directly connect to user_db**.

Similarly:





5. Check database ownership

Verify:

User Service → user_db 

Order Service → order_db 

Payment Service → payment_db 

Architecture Testing:

Tested the User, Order, and Payment services and verified their REST APIs and database operations. Confirmed that each service accesses only its own database.

Verified service-to-service communication through APIs and ensured that no service directly accesses another service's database.

Also verified successful creation and retrieval of users, orders, and payments.

Step 9 — Document Common Problems

1. Duplicate Data

Sometimes the same information may need to exist in multiple services.

Example:

User Service → user_db

Order Service → order_db

The Order Service might store userId and some user information needed for orders.

Problem: If the user's information changes, the copied information may become outdated.

Solution: Keep only the required data and use APIs/events to obtain the latest information when necessary.

2. Data Synchronization

Because each service has its own database, keeping related information synchronized can be difficult.

Example:

User Service



User changes email



Order Service may have old information

Solution: Use REST APIs or asynchronous events/messages to communicate changes between services.

3. Distributed Transactions

A business operation may involve more than one service/database.

Example:

Create Order



Order Service → order_db



Payment Service → payment_db

If the payment fails after the order is created, you need to handle that situation.

Solution: Use patterns such as **Saga**, compensation actions, and reliable messaging rather than trying to use one database transaction across all services.

Common Problems in Database-per-Service Architecture:

1. **Duplicate Data:** Some data may need to be duplicated between services, which can cause stale or inconsistent information.
2. **Data Synchronization:** Changes in one service may need to be communicated to other services to keep related data consistent.
3. **Distributed Transactions:** A single business operation may involve multiple services and databases, making transaction management more complex. Saga and compensation mechanisms can be used to handle failures.

Conclusion: Each service should own its data, communicate through APIs/events, and use appropriate synchronization and transaction strategies.

